

RAPPORT DE PROJET DE PROGRAMMATION IMPÉRATIVE

*Traitements d'images aux formats
PPM et PGM*

William CHEYMOL

• Projet encadré par Guillaume BUREL •



Table des Matières

Contexte

Photoshop, Gimp, Photofiltre, ... De nos jours il existe des dizaines, même des centaines de logiciels permettant de faire des traitements sur des images. Vous en avez forcément manipulé, et sûrement aujourd'hui vous trouvez leur fonctionnement totalement banal. Pourtant, le moindre traitement que vous choisissiez d'effectuer engendre des dizaines et des dizaines de calculs afin le résultat puisse apparaître sur votre écran en une fraction de secondes.

Dans ce projet, nous allons explorer ces mécanismes en profondeur. À travers les formats d'images simples comme *PGM* (niveaux de gris) et *PPM* (couleurs), nous considérerons les images comme ce qu'elles sont - à savoir des tableaux de pixels - pour réimplémenter, en C, certains de ces traitements que nous avons l'habitude de voir en un clic.

Structures de données

Un des premiers choix effectués a été de séparer les structures selon le type de pixel que l'on manipule. Cela permet une plus grande flexibilité et surtout évite des erreurs d'indixages, pouvant se révéler très frustrantes et chronophages (croyez - moi j'en ai fait les frais) .

```
typedef struct pixel_ppm {
    byte components[3];
} pixel_ppm;

typedef struct pixel_pgm{
    byte intensity;
} pixel_pgm;
```

Dans le module pixel.[h|c], nous avons donc défini en plus de ces structures, des fonctions permettant d'accéder et d'assigner de nouvelles valeurs à aux valeurs des composantes de chaque type de pixel. Le type byte correspond à un *unsigned char*, c'est - à - dire un entier non signé prenant des valeurs entre 0 et 255. Cela est plus simple à manipuler pour des images, notamment pour la lecture de celles - ci.

Après cela, nous avons pu aisément définir une structure pour nos images :

```
typedef struct picture{
    unsigned int width;
    unsigned int height;
    unsigned int channels;
    void *data;
} picture;
```

Nous avons également défini une structure LUT (Look Up Table, cf. 3.7) afin de pouvoir les manipuler facilement :

```
struct lut {
    unsigned int  taille;
    byte*  table;
};
```

Fonctionnement de notre programme

Lecture et écriture de fichiers images

La première chose que nous devons faire, avant de pouvoir manipuler les images, c'est de créer une fonction prenant en paramètre un nom de fichier, par exemple :

C : /Users/machinbidule/OneDrive/Documents/Images/Tulipscolors.ppmL

et qui retransmet les données binaire de l'image dans une structure *picture* que nous venons de créer, contenant ainsi la largeur, la hauteur, le nombre de canaux (1 si image au format *PGM*, 3 si image au format *PPM*) ainsi que l'intensité de chaque composante de pixel.

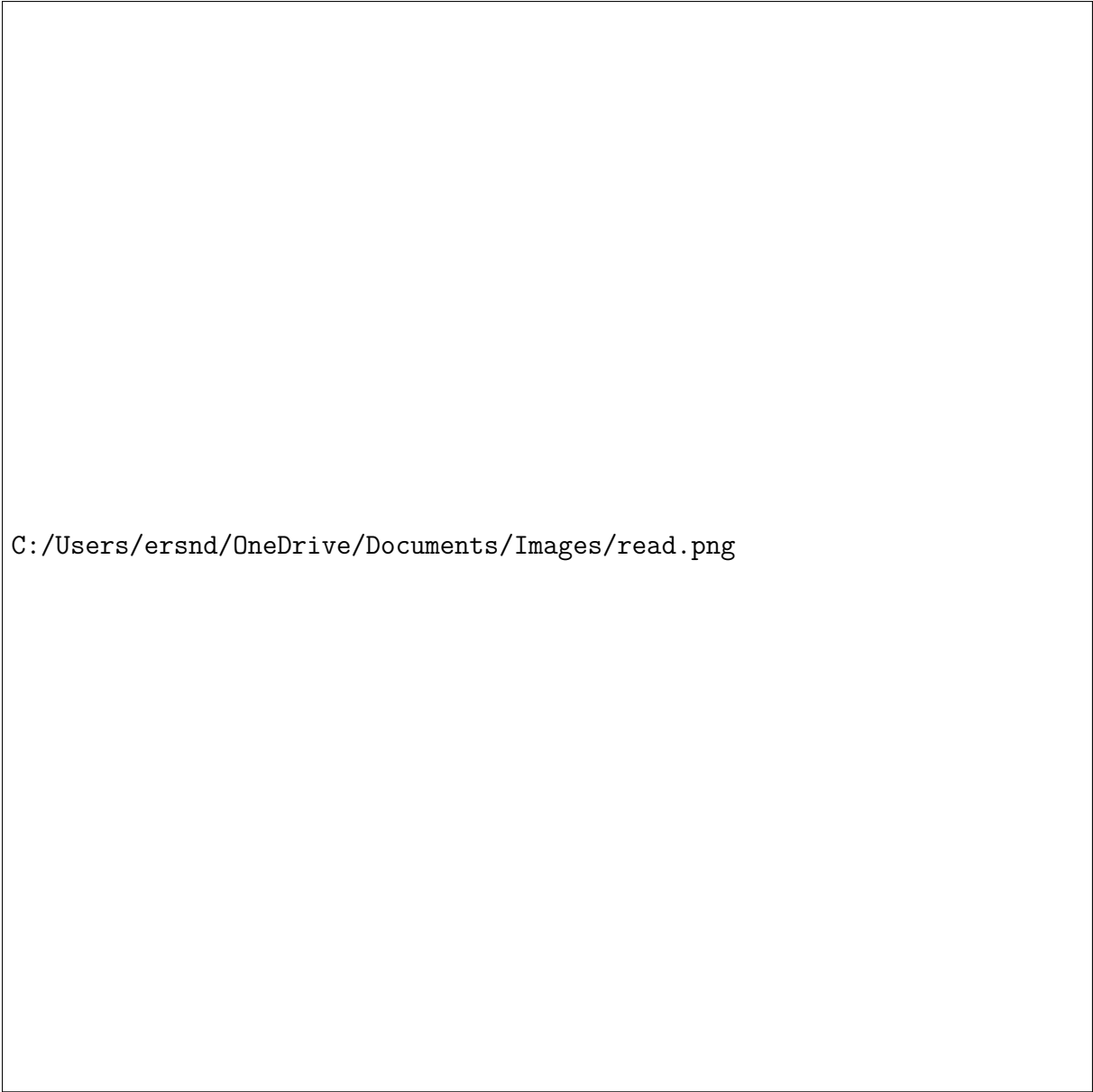
De même, il nous faut une fonction *write* permettant le processus inverse : prenant en argument un nouveau de fichier et une structure *picture* , et créé un fichier PPM/PGM avec les données codées de nouveau en binaire reprenant les informations de l'image donnée en argument :

L'implémentation se fait donc de cette manière :

```
picture read_picture(const char* path);
int write_picture(picture p, const char* filename);
```

Pour la fonction *read*, on avance par étapes:

- On commence par lire le début du fichier : le numéro magique correspondant au format de l'image, les dimensions, etc. ;
- On fait des vérifications sur ces premières données ;
- On lit le reste du fichier avec *fread* pour convertir les données binaires en des données correspondant aux intensités des pixels ;
- On vérifie qu'on a bien tout lu, qu'il n'y a ni trop ni pas assez de pixels.



`C:/Users/ersnd/OneDrive/Documents/Images/read.png`

Figure 1: Principe de fonctionnement des fonctions *read* et *write*

Pour la fonction *write*, on exécute le processus dans le sens inverse :

- On récupère les informations de la structure ;
- On écrit le début du fichier (numéro magique, dimensions, etc.) ;
- On réécrit les données en binaire avec *fwrite* .

Ces fonctions vont nous permettre pour toute la suite du projet de pouvoir manipuler efficacement les images en récupérant ensuite simplement l'intensité de nos pixels grâce aux fonctions suivantes, présentes dans notre module `pixel.h|c` :

```
byte get_pixel_ppm_component_value(pixel_ppm* p, enum  
color c);
```

```

void set_pixel_ppm_component_value(pixel_ppm* p, enum
color c, byte value);
byte get_pixel_pgm_value(pixel_pgm* p);
void set_pixel_pgm_value(pixel_pgm* p, byte value);

```

Les *getter* permettent de récupérer les intensités de composantes de pixels tandis que les *setter* permettent de modifier leurs valeurs.

Gestion des images

Différentes fonctions ont été codées dans cette partie afin de manipuler les images, désormais sous la forme de notre structure *picture* :

```

picture create_picture(unsigned int width, unsigned int
height, unsigned int channels);
void clean_picture(picture* p);
picture copy_picture(picture p);
void info_picture(picture p);
int is_empty_picture(picture p);
int is_gray_picture(picture p);
int is_color_picture(picture p);

```

Ces premières fonctions nous permettent de manipuler la structure d'images : création, nettoyage, copie, affichage des caractéristiques et d'autres fonctions permettant de vérifier le format de nos images.

D'autres fonctions plus poussées permettent déjà de faire quelques manipulations pour nos images, comme passer d'un format à un autre. On peut également séparer une image PPM en ses trois composantes rouge, verte et bleue ou même recomposer une image PPM à partir de 3 composantes :

```

picture convert_to_color_picture(picture p);
picture convert_to_grey_picture(picture p);
picture* split_picture(picture p);
picture merge_picture(picture red, picture green,
picture blue);

```

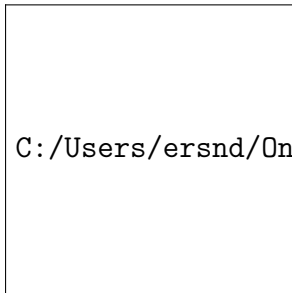
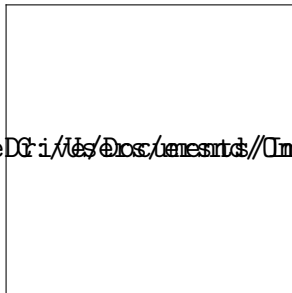
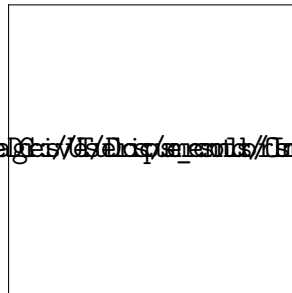


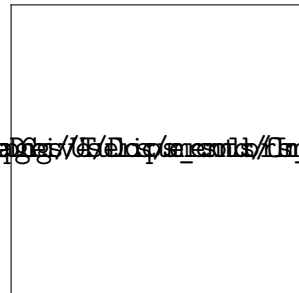
Image originale



Composante rouge



Composante verte



composante bleue

Toutes ces fonctions reposent juste sur la manipulation de pixels à partir de l'image entrée.

Manipulation directe des valeurs des pixels

À partir des fonctions précédentes, nous avons maintenant tout ce qu'il faut sous la main pour coder des fonctions afin de faire des traitements plus avancés.

Le premier traitement que nous pouvons exécuter est un éclaircissement à partir d'un certain facteur :

```

picture brighten_picture(picture p, double factor);

```

Cette fonction multiplie chaque valeur d'une composante de pixel par le facteur donné et renvoie le minimum entre ce calcul et 255, 255 étant la valeur maximale d'un *unsigned char*.

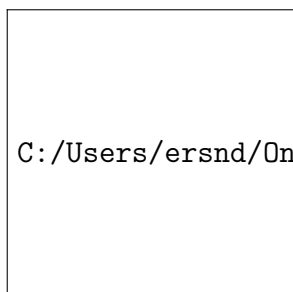


Image originale

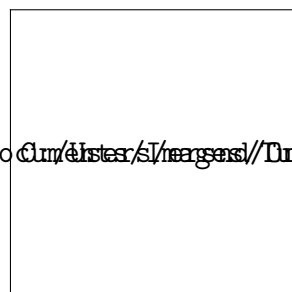


Image éclaircie d'un
facteur 1.5

Il est également possible de faire fondre une image (ici vers le bas) : on choisit un nombre *number* de pixels au hasard dans l'image et si le pixel au dessus est plus sombre, alors le pixel original prend est remplacé par ce dernier.

```
picture melt_picture(picture p, int number);
```

On remarquera que l'effet fondu pour une itération n'est pas très bien remarquable, mais que pour une petite dizaine d'itérations celui - ci est bien plus marqué.

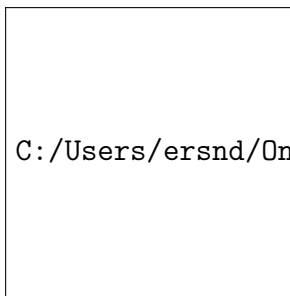
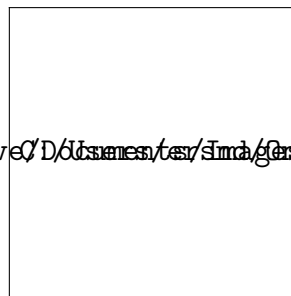
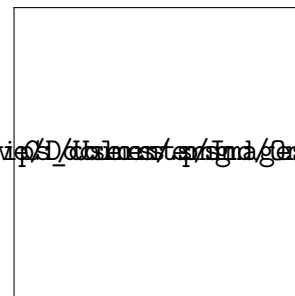


Image originale



Après une itération



Après 10 itérations

Opérations arithmétiques sur les images

Nous avons vu qu'il était possible d'effectuer quelques opérations sur les pixels à partir de paramètres extérieurs. Maintenant nous allons voir comment nous pouvons utiliser ces opérations sur les pixels entre eux.

La première chose que nous pouvons faire est de créer une différence en valeur absolue entre deux images :

```
picture diff_picture(picture p1, picture p2);
```

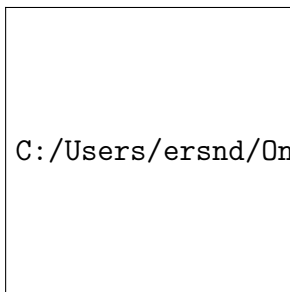


Image 1

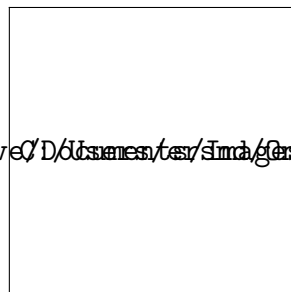
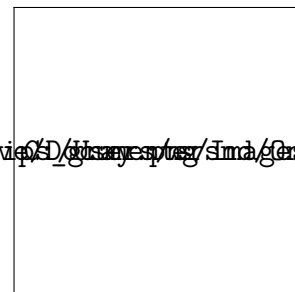


Image 2



Soustraction entre
l'image 1 et l'image 2

Il est également possible de multiplier les valeurs entre elles. Le tout se fait évidemment avec un modulo 256 afin de rester dans la plage de valeurs autorisée pour chaque composante :

```
picture mult_picture(picture p1, picture p2);
```

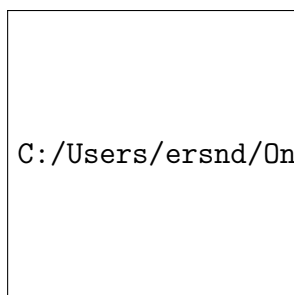


Image 1

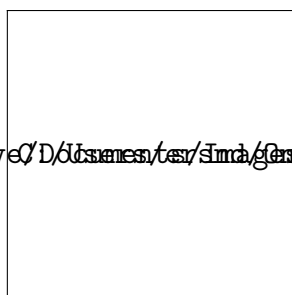
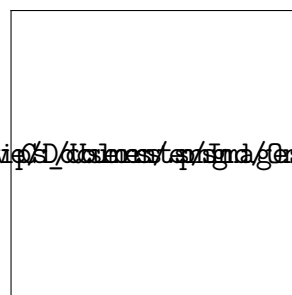


Image 2



Produit entre l'image
1 et l'image 2

Finalement, il est possible de mélanger deux images entre elles à partir d'un masque :

```
picture mix_picture(picture p1, picture p2, picture p3);
```

Ce qu'il se passe avec cette fonction, c'est que l'intensité de la couleur de la première image sur le résultat va dépendre de $\alpha = \frac{\gamma}{255}$, tandis que l'intensité de la couleur de la seconde image dépendra directement de $1 - \alpha$, où $\gamma = \text{intensité du masque à ce pixel}$. Voyez plutôt :

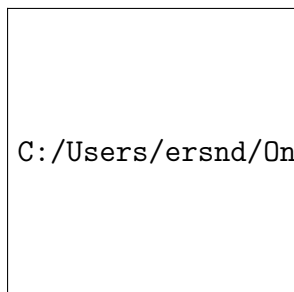


Image 1

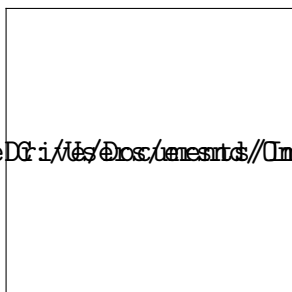
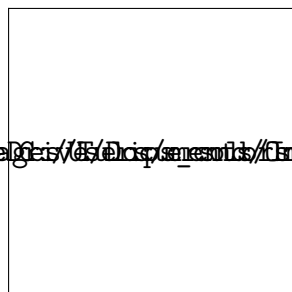
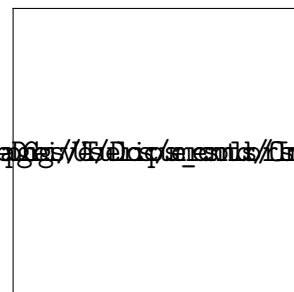


Image 2



Masque



Résultat final

Ré-échantillonnage d'images

Le redimensionnement d'image est une autre problématique intéressante sur laquelle nous nous sommes penchés. Car redimensionner une image, ce n'est pas aussi facile que cela en a l'air. Nous avons utilisé deux politiques différentes pour redimensionner une image.

La première s'appelle **la politique du plus proche voisin**. Le principe est très simple : lorsque nous redimensionnons l'image, nous cherchons quel est le plus proche voisin d'un nouveau pixel dans la grille de base (*voir ci - dessous*) . Après cela, nous donnons à ce pixel la valeur de "*son plus proche voisin*".



Figure 2: Procédés de redimensionnement

Une deuxième méthode de redimensionnement, nommée **politique d'interpolation bilinéaire**, consiste à se fier non pas seulement sur un pixel, mais bien sur les 4 pixels entourant le pixel original : P1, P2, P3 et P4. La valeur du pixel est après cela calculée à partir de la formule suivante :

$$P_{new} = (1 - \alpha)(1 - \beta)P_1 + \alpha(1 - \beta)P_2 + (1 - \alpha)\beta P_3 + \alpha\beta P_4$$

avec α et β les coefficients d'interpolation horizontale et verticale.

```
picture resample_picture_nearest(picture image, unsigned
int width, unsigned int height);
picture resample_picture_nearest(picture image, unsigned
int width, unsigned int height);
```

Ces fonctions ne sont pas si faciles que cela à implémenter et méritent que l'on si penche de plus près.

La première chose à faire est de calculer le rapport d'agrandissement / de rétrécissement de notre image :

```
float resize_ratio_x = (float)p.width / width;  
float resize_ratio_y = (float)p.height / height;
```

Après cela, il nous faut calculer les coordonnées x et y de notre pixel dans l'image entrée en argument (i représente l'indice de notre pixel dans le tableau de données de l'image) :

```
int x = i%width;  
int y = i/width;
```

Finalement, on peut calculer les nouvelles coordonnées de notre pixel :

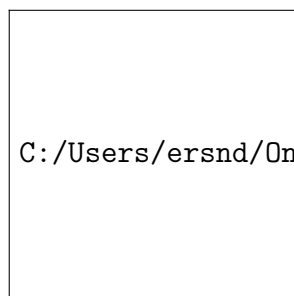
```
float nix = (x*size_ratio_x);  
float niy = (y*size_ratio_y);
```

Et après cela, on peut faire ce que l'on veut :

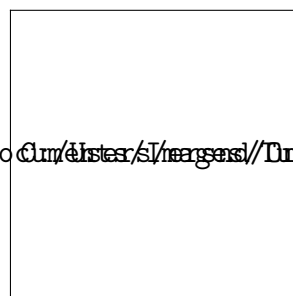
- Soit on simule un arrondi classique et ajoutant $+0.5$ et en convertissant en entier pour trouver les coordonnées du pixel "le plus proche" .
- Soit on prend les coordonnées des 4 pixels voisins.

Ce sont donc ces procédés de calcul qui nous ont permis de coder les fonctions de redimensionnement.

Nous pouvons d'ailleurs comparer ces deux méthodes en utilisant la fonction *diff* codée auparavant ainsi que la fonction *normalize* que vous trouverez au paragraphe suivant :



Pour une image en
niveaux de gris



Pour une image en
couleurs

Manipulation des valeurs des pixels en utilisant une LUT (Look Up Table)

Une autre manière de manier les pixels sans faire les calculs directement est d'utiliser une LUT (Look Up Table) - aussi appelée fonction de transfert, qui va assimiler de nouvelles valeurs à nos pixels. Par exemple, en utilisant la LUT identité, on crée simplement une copie de notre image :



Figure 3: LUT Identité

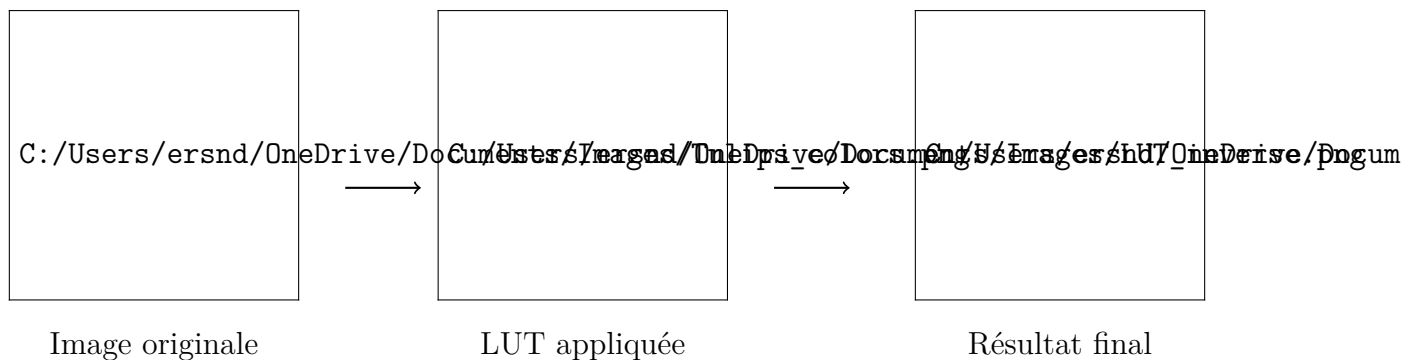
Nous avons pu coder 4 fonctions utilisant des LUT afin de faire encore plus de traitements d'images :

```
picture inverse_picture(picture p);  
picture normalize_dynamic_picture(picture p);  
picture set_levels(picture p, byte nb_levels);  
picture picture_brighten_lut(picture p, double factor);
```

Tous ces traitements fonctionnent à l'aide de notre module lut.[h|c] dans lequel nous avons défini le type lut et écrit de nombreuses fonctions permettant de manipuler des LUT, dont notamment cette dernière, qui permet d'appliquer une LUT à une image :

```
void apply_lut(picture* p, lut l);
```

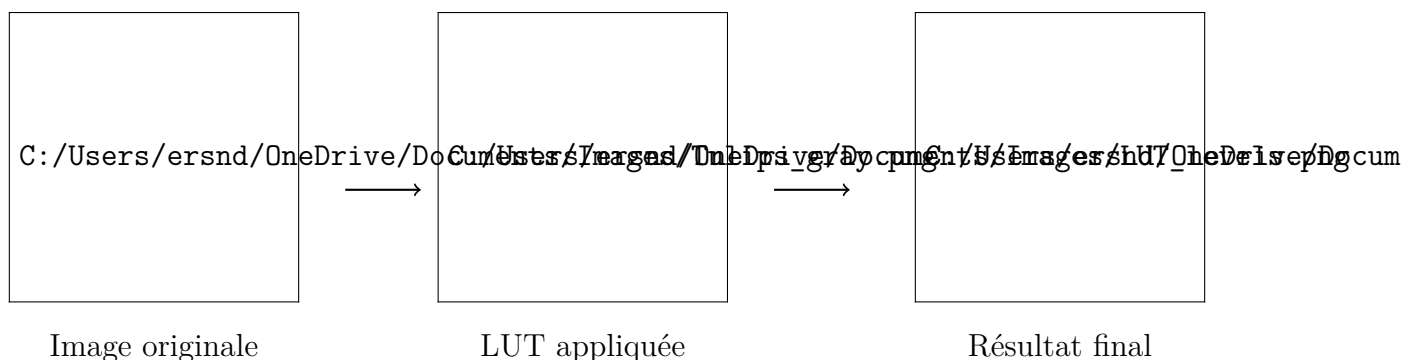
Ainsi, la première fonction que nous avons pu coder applique la LUT suivante à une image pour l'inverser :



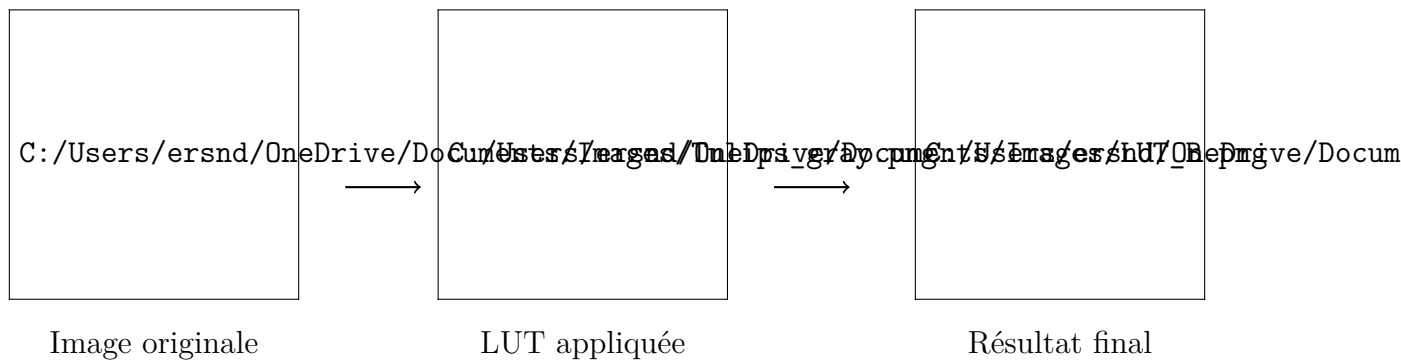
La fonction *normalize*, quant à elle, prend une image PGM et la normalise, c'est - à - dire qu'elle s'assure que l'image final contienne bien toutes les nuances d'intensité disponibles. Pour le cas des images PPM, le principe est très simple : on décompose notre image avec la fonction *split*, on normalise chacune de ses composantes puis on reforme l'image finale grâce à *merge*.



Il est également possible de limite le nombre de valeurs possibles pour l'intensité de nos pixels : Celle - ci est en effet réglée de base à 256, mais il est possible par exemple de la réduire à 8 en utilisant une LUT dite "*en escalier*" .



Finalement, nous pouvons également créer une LUT, qui peut permettre de reconstruire la fonction *brighten*, qui rappelons - le éclaircit l'image d'un certain facteur.



Problèmes rencontrés et solutions trouvées

Peu de problèmes ont été rencontrés concernant la manipulation des images en elles - même, les fonctions à coder n'étant peu complexes, sauf celles concernant le redimensionnement nécessitant un peu plus de réflexion. En revanche, il a été plus difficile de coder la fonction *read*. Celle - ci nécessitait beaucoup plus de réflexion, notamment pour certains cas plus difficiles à gérer que d'autres.

Tout d'abord, au niveau de la lecture de l'image en binaire, il a été difficile de gérer cette lecture correctement, notamment à cause d'erreur d'indicages, ce qui a rapidement poussé à créer deux types de structures différentes pour les pixels PPM et les pixels PGM. Même après cela, il a été indispensable de modifier le code suivant par en introduisant un compteur global pour la lecture de nos pixels :

```

while ((n = fread(buffer, 1, bloc_size, file)) > 0) {
    bytes_lus += n;
    size_t i;
    for (i = 0; i < n / 3; i++) {
        pixelptr.components[0] = buffer[i * 3];
        pixelptr.components[1] = buffer[i * 3 + 1];
        pixelptr.components[2] = buffer[i * 3 + 2];
        pixelptr++;
    }
}
  
```

Code original, qui menait à l'erreur qui suit (*cf* page suivante) . *pixelptr* correspond au pointeur pour les pixels dans notre lecture en binaire

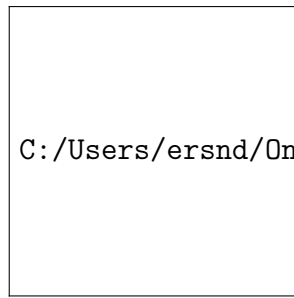


Image à lire

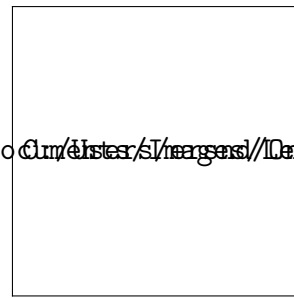


Image lue avec le
code erroné ci -
dessus

La correction de cette erreur repose, comme expliqué par l'introduction d'un compteur global, ici *pixelindex*, afin de manipuler plus facilement les pixels :

```
pixel_ppm *pixelptr = (pixel_ppm*)picres.data;

size_t pixelindex = 0;
byte value;
while ((n = fread(buffer, 1, bloc_size, file)) >
0) {
    bytes_lus+=n;
    for (size_t i = 0; i < n; ++i) {
        value = buffer[i];
        enum color current_color;
        if (pixelindex % 3 == 0)
            current_color = RED;
        else if (pixelindex % 3 == 1)
            current_color = GREEN;
        else
            current_color = BLUE;
        set_pixel_ppm_component_value(((
        pixel_ppm*)picres.data) + (pixelindex /
        3), current_color, (byte)(value*
        correction));
        pixelindex++;
    }
}
```

Pour tester la robustesse de cette fonction, nous avons également eu besoin de coder une procédure *commentaries*, qui comme son nom l'indique permet de sauter les commentaires de notre fichier non encore converti en structure. Cette procédure a causé beaucoup de problèmes afin que son fonctionnement soit correct. Notamment, il a été difficile de gérer le pointeur correctement afin que les bonnes lignes soient sautées. Finalement, c'est le code suivant qui a permis de s'en sortir : si une ligne vide ou commençant par '#' est rencontrée, on la lit et on garde la position et on passe à la ligne suivante. Si une ligne correcte est rencontrée, on revient à la position sauvegardée :

```

void commentaries(FILE* file) {
    long position;
    char line[256];
    while (fgets(line, sizeof(line), file)) {
        if ((unsigned char)line[0] == 35 || (
            unsigned char)line[0] == 10) {
            position = ftell(file);
            continue;
        } else {
            fseek(file, position, SEEK_SET);
            break;
        }
    }
}

```

Conclusion

L'ouverture, la lecture et le traitement de fichiers PPM et PGM restent intéressants en raison de la simplicité de leur structure. Comme le montre notre programme, ces formats constituent une base solide pour apprendre les fondamentaux du traitement d'images. Cependant, pour des traitements plus avancés, il serait nécessaire de prendre en charge des formats plus courants comme JPEG ou PNG. De plus, les fonctions que nous avons développées ici, bien qu'efficaces pour des images de taille modérée, peuvent se révéler coûteuses en termes de mémoire et de calcul lorsqu'elles sont appliquées à des images volumineuses ou de manière répétée. Par exemple, la fonction *melt*, bien qu'opérationnelle, montre ses limites lorsqu'elle est itérée plus d'une dizaine de fois, entraînant un ralentissement significatif du programme.